



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 2C: Process: Hardware vs. Simulation

Processes: Hardware vs. Sim



- Hardware: Concept of “process” invisible to synthesizer
 - A process will always “run” in hardware
 - Synthesizer ignores “process” keyword and sensitivity list
 - Statements in the process are concurrent when considering hardware

Processes: Hardware vs. Sim Continued



- Simulation: Process executes when any of the objects in the sensitivity list “change” value
 - Statements in the process are executed in order and updated at the end of the process

Hardware vs. Simulator I

- Can we write a CSA as a process?

```
S <= A XOR B XOR Ci;  
Co <= (A AND B) OR (B AND Ci) OR (A AND Ci);
```

Do both set of statements behave the same?

Hardware vs. Simulation:

Both behave the same in hardware (on FPGA)

Both behave the same in simulation

```
PROCESS (A, B, Ci)  
BEGIN  
    S <= A XOR B XOR Ci;  
    Co <= (A AND B) OR (B AND Ci) OR (A AND Ci);  
END PROCESS;
```

Hardware vs. Sim II

```
S <= A XOR B XOR Ci;  
Co <= (A AND B) OR (B AND Ci) OR (A AND Ci);
```

Do both set of statements behave the same in Simulation?

NOT the same in simulation

Do both behave the same in hardware (on FPGA)?

Yes... but you will get warnings **during synthesis**

```
PROCESS (B, Ci)  
BEGIN  
    S <= A XOR B XOR Ci;  
    Co <= (A AND B) OR (B AND Ci) OR (A AND Ci);  
END PROCESS;
```

Simulation and Simulators - Example

- Testbench – an empty entity (no ports)

```
library IEEE;
use ieee.std_logic_1164.all;
use IEEE.numeric_std.all;
use work.all;
```

```
entity test_TB is      -- empty entity
end test_TB;
```

```
architecture TB of test_TB is
```

```
signal A1, B1, Ci1: std_logic;
signal S1, Co1, S2, Co2 : std_logic;
```

```
Begin
```

```
U_example: entity addbit port map (A1, B1, Ci1, S1, Co1);
U_example2: entity addbit2 port map (A1, B1, Ci1, S2, Co2);
```

```
A1 <= '0', '1' after 20 ns, '0' after 40 ns, '1' after 60 ns, '0' after 80 ns,
      '1' after 100 ns, '0' after 120 ns, '1' after 140 ns;
B1 <= '0', '1' after 40 ns, '0' after 80 ns, '1' after 120 ns;
Ci1 <= '0', '1' after 80 ns;
```

```
end TB;
```

Simulation and Simulators - UUT

- Design with **process**

```
library ieee;
use ieee.std_logic_1164.all;
use work.all;

ENTITY addbit IS PORT(
    A, B, Ci : IN std_logic;
    S, Co1: OUT std_logic);
END addbit;

ARCHITECTURE struct OF addbit IS
BEGIN
    PROCESS (A, B, Ci)
    BEGIN
        S <= A XOR B XOR Ci;
        Co1 <= (A AND B) OR
                (B AND Ci) OR
                (A AND Ci);
    END PROCESS;
END struct;
```

- Design **without** process

```
library ieee;
use ieee.std_logic_1164.all;
use work.all;

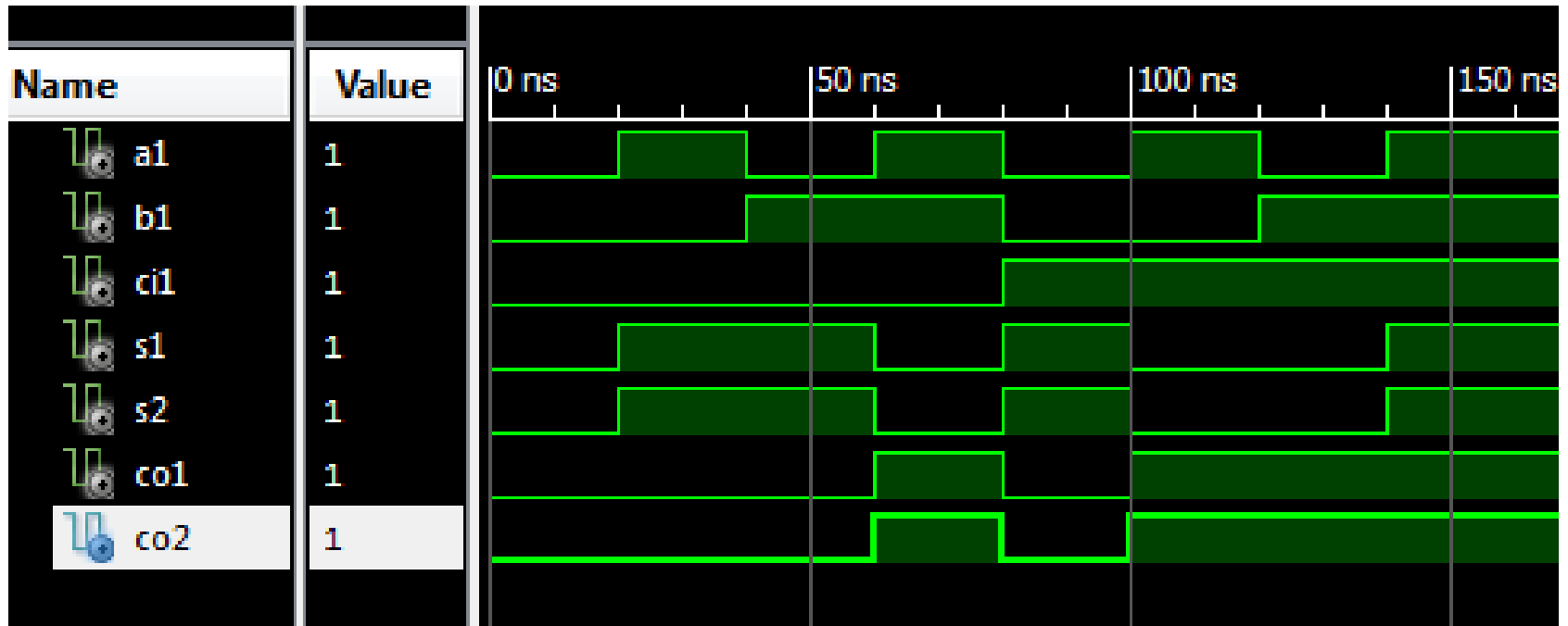
ENTITY addbit2 IS PORT(
    A, B, Ci : IN std_logic;
    S, Co2: OUT std_logic);
END addbit2;

ARCHITECTURE struct OF addbit2 IS
BEGIN

    S <= A XOR B XOR Ci;
    Co2 <= (A AND B) OR
            (B AND Ci) OR
            (A AND Ci);

END struct;
```

Simulation and Simulators - Results



Simulation and Simulators – UUT II

- Design with **process**

```
library ieee;
use ieee.std_logic_1164.all;
use work.all;

ENTITY addbit IS PORT(
    A, B, Ci : IN std_logic;
    S, Co1: OUT std_logic);
END addbit;

ARCHITECTURE struct OF addbit IS
BEGIN
    PROCESS (      B, Ci)
    BEGIN
        S <= A XOR B XOR Ci;
        Co1 <= (A AND B) OR
                (B AND Ci) OR
                (A AND Ci);

    END PROCESS;
END struct;
```

- Design **without** process

```
library ieee;
use ieee.std_logic_1164.all;
use work.all;

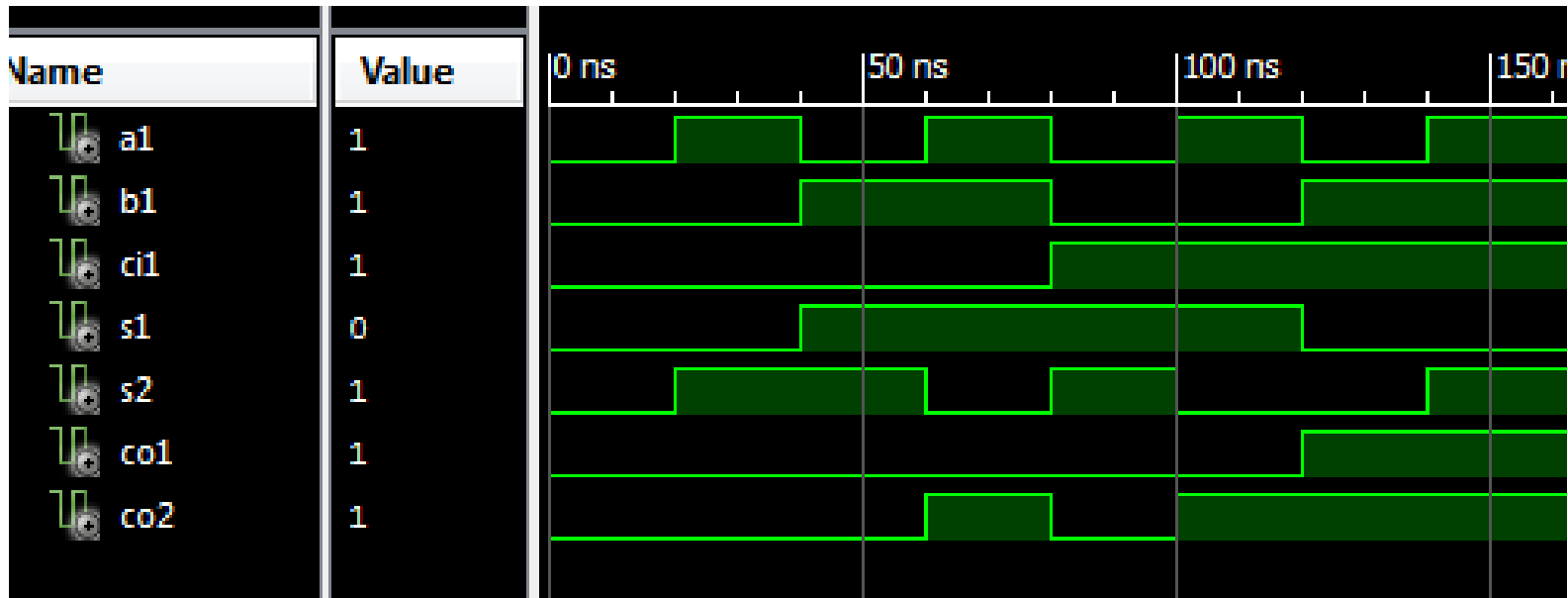
ENTITY addbit2 IS PORT(
    A, B, Ci : IN std_logic;
    S, Co2: OUT std_logic);
END addbit2;

ARCHITECTURE struct OF addbit2 IS
BEGIN

    S <= A XOR B XOR Ci;
    Co2 <= (A AND B) OR
            (B AND Ci) OR
            (A AND Ci);

END struct;
```

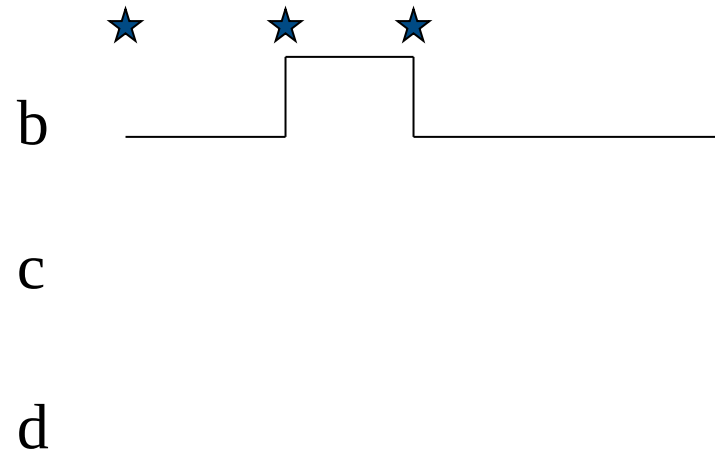
Simulation and Simulators – Results II



- More on this to come in later slides.
 - But for now, be careful...
 - Simulations may not always match hardware

Process Examples: Simulator

```
example : process (b)
begin
    c <= not b;
    d <= not c;
    c <= b;
    c <= not b;
end process example;
```



- Potential Quiz question

Simulation - Testbenches

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.all;
```

```
entity testProcess is
end testProcess;
```

```
architecture testbench of testProcess is
```

```
signal b, c, d : std_logic;
```

```
begin
```

 **b <= ; -- generate the waveform for b to match the waveform in the previous slide**

```
-- Only this section of the VHDL code will be repeated in the quiz
```

```
example : process(b)
```

```
begin
```

```
    c <= not b;
```

```
    d <= not c;
```

```
    c <= b;
```

```
    c <= not b;
```

```
end process example;
```

```
end testbench;
```

Processes: Hardware vs. Sim Reflection



- Q: Why are we discussing this?
- A: Good VHDL design practices will result in simulations that match hardware
- Make sure you are using good design practices when designing VHDL!